

```
=====
1. What is a Program?
=====
```

**** A program is a set of instructions written to perform a particular task.**

Examples:

Calculator program
Banking application
YouTube application
File downloader
Chat application

**** When we execute a Python program, the operating system creates a process.**

```
=====
2. What is a Process?
=====
```

**** A process is a program that is currently running.**

For example, when you open:

Google Chrome
PyCharm
Visual Studio Code
Spotify
Zoom
Notepad

**** each application runs as a separate process.**

A process has its own:

Memory
Variables
Resources
Execution environment

```
=====
## What is a Thread?
=====
```

**** A thread is a small unit of execution inside a program/process.**

**** A single process can contain one or more threads.**

=> For example, consider a web browser.

=> A browser may use separate threads for:

**** Loading web page**

**** Playing video**

**** Download a file**

**** Checking user input**

**** Display animations**

=> All these activities happen inside the same browser process.

=====
Process vs Thread
=====

**** Process**

- A process is a running program
- Every process will have separate memory
- Creating process is expensive
- Communication between process is slower
- One process can contain multiple threads

**** Thread**

- A thread is a unit of execution inside a process
- Threads share the process memory
- Creating a thread is lightweight
- Communication between threads is faster
- A thread always belongs to a process

=====
What is Multitasking?
=====

**** Multitasking means performing multiple tasks during the same period.**

**** There are two major types of multitasking:**

1) Process Based Multi Tasking

**** Multiple programs run at the same time**

- Playing music
- Writing code in pycharm
- Browsing internet

**** Each application usually runs as separate process.**

2) Thread Based Multi Tasking

**** Multiple tasks run inside the same program.**

Example:

A Python application may:

- Download a file
- Show progress
- Receive user input
- Save information to a database

**** These tasks can be handled using different threads.**

```
=====
What is Multithreading
=====
```

**** Multi-threading means executing multiple threads within the same process.**

**** In Python, multithreading is mainly used when a program has multiple independent tasks, especially tasks that spend time waiting.**

```
-- Downloading multiple files
-- Calling Multiple REST APIs
-- Reading Multiple files
-- Database operations
-- Send Emails
-- Chat Applications
-- Network communications
```

```
=====
Real-Life Example
=====
```

Imagine a restaurant with only one employee.

The employee must:

```
Take the customer's order
Prepare the food
Serve the food
Collect payment
```

The second customer must wait until all the work for the first customer is completed.

This is similar to single-threaded execution.

Now imagine the restaurant has multiple employees:

```
employee-1 takes orders
employee-2 prepares food
employee-3 serves food
employee-4 collects payments
```

Multiple activities can happen at the same time. This is similar to Multi Threading.

**** Multiple tables are served during the same period. This is similar to multithreading.**

**** Each employee represents one thread.**

```
=====
Main Thread
=====
```

**** Every Python program starts with one default thread called the main thread.**

```
** Python provides the "threading" module for multithreading.
```

```
---
```

```
import threading
```

```
print("Current Thread:", threading.current_thread())
print("Thread Name:", threading.current_thread().name)
print("Thread Identifier:", threading.current_thread().ident)
print("Active Thread Count:", threading.active_count())
```

```
----
```

```
=====
Creating Threads in Python
=====
```

```
** Python provides the "threading" module for multithreading.
```

```
** We need to "import threading" module for multi threading...
```

```
** A thread can be created using the "Thread" class.
```

Syntax :

```
thread = threading.Thread(target=function_name)
```

Here

```
-- Thread creates a new thread.
-- target specifies the function the thread should execute.
-- start () method starts the new thread (worker thread)
```

Ex--1:

```
import threading
import time
```

```
def task_one():
    print("Task-1 Started")
    time.sleep(3)
    print("Task-1 Completed")
```

```
def task_two():
    print("Task-2 Started")
    time.sleep(2)
    print("Task-2 Completed")
```

```
thread1 = threading.Thread(target=task_one)
thread2 = threading.Thread(target=task_two)
```

```
thread1.start()
thread2.start()
```

```
##### Another way of creating thread #####
```

```
class MyThread(threading.Thread):
```

```

def run(self):
    print("Run() method started..")
    print("Thread Name :", threading.current_thread().name)

thread3 = MyThread()
thread3.start()

```

Note-1 : we are overriding run() method to provide our own implementation.

Note-2: When we call start() method internally it executes run() method.

```

=====
Important Thread Methods
=====

```

- 1) start()
- 2) join()
- 3) is_alive()

```

=====
what is start( )
=====

```

** The start() method begins the execution of a thread.

```
t1.start ( )
```

** Python internally calls the target function in a separate thread.

** Do not call the run() method directly when you want separate thread execution.

```
t1.run ( )
```

Note : Calling run() directly executes the function using the current thread. It does not start a new thread.

```

=====
what is join( )
=====

```

** The join() method makes the current thread wait until another thread completes its execution.

Scenario :

Thread-1 : Download a file

Thread-2 : Read data from downloaded file

Example :

```

def download_file():
    print("Download file started..")
    time.sleep(3)
    print("Download completed")

t5 = threading.Thread(target=download_file)
t5.start()

```

```
t5.join()

print("Read data from downloaded file...")
```

Note: Without join(), the main thread may continue before the worker thread finishes.

```
=====
what is is_alive( )
=====
```

**** The is_alive() method checks whether a thread is currently running.**

Example :

```
def process_data():
    time.sleep(3)

thread = threading.Thread(target=process_data)

thread.start()
print(thread.is_alive())

thread.join()

print(thread.is_alive())
```

```
=====
Thread Scheduling
=====
```

**** When multiple threads are available, the operating system decides:**

- which thread should execute
- how long it should execute
- when execution should switch to another thread

**** This process is called thread scheduling.**

**** Because scheduling is controlled by the operating system, thread execution order is not guaranteed.**

```
=====
What is Race Condition
=====
```

**** When multiple threads try to access and modify shared data at the same time may produce an incorrect result which is called as Race Condition.**

Ex :

Consider a bank account.

Two threads are trying to withdraw money from the same account at the same.

```
-----
```

```
import threading
import time
```

```
balance = 1000
```

```
def withdraw(amount):
    global balance
    if balance >= amount:
        current_balance = balance
        time.sleep(1)
        balance = current_balance - amount
        print("Withdraw successfully")
        print("Remaining Balance:", balance)
    else:
        print("Insufficient Balance")

t1 = threading.Thread(target=withdraw, args=(700,))
t2 = threading.Thread(target=withdraw, args=(700,))

t1.start()
t2.start()

t1.join()
t2.join()

print("Final Balance :", balance)
```

**** Both threads may read the original balance as 1000.**

**** Both threads may then allow a withdrawal of 700.**

**** This creates incorrect account processing.**

**** The exact result depends on thread scheduling, which makes race-condition bugs difficult to reproduce.**

=====

What is Thread Synchronization

=====

**** Thread synchronization means controlling multiple threads so that only one thread accesses a critical shared resource at a time.**

**** Python provides synchronization tools such as:**

- Lock
- RLock
- Semaphore
- Event
- Condition

Note: For beginners, the most important tool is Lock.

**** A Lock allows only one thread at a time to execute a protected section of code.**

```
lock = threading.Lock()
```

**** Before accesing shared resource we need to acquire lock like below**

```
lock.acquire()
```

**** After completing the work we need to release the lock like below**

```
        lock.release()
```

```
-----

import threading
import time

balance = 1000

lock = threading.Lock()

def withdraw(amount):
    global balance

    lock.acquire()

    try:
        if balance >= amount:
            print(
                threading.current_thread().name,
                "is processing withdrawl"
            )

            current_balance = balance
            time.sleep(1)
            balance = current_balance - amount
            print("Remaining Balance:", balance)
        else:
            print("insufficient balance")
    finally:
        lock.release()

t1 = threading.Thread(target=withdraw, args=(700,), name="Customer-1")
t2 = threading.Thread(target=withdraw, args=(700,), name="Customer-2")

t1.start()
t2.start()

t1.join()
t2.join()

print("Final Balance:", balance)
```

```
=====
implement Lock by using with keyword
=====
```

**** The recommended way to use a lock is with the "with" statement.**

Syntax :

```
with lock:
    # Critical section
```

**** Python automatically acquires and releases the lock.**

```
-----

import threading
import time

balance = 1000

lock = threading.Lock()

def withdraw(amount):
    global balance

    with lock:
        if balance >= amount:
            print(
                threading.current_thread().name,
                "is processing withdrawl"
            )
            current_balance = balance
            time.sleep(1)
            balance = current_balance - amount
            print("Remaining Balance:", balance)
        else:
            print( threading.current_thread().name, "- Insufficient balance" )

t1 = threading.Thread(target=withdraw, args=(700,), name="Customer-1")
t2 = threading.Thread(target=withdraw, args=(700,), name="Customer-2")

t1.start()
t2.start()

t1.join()
t2.join()

print("Final Balance:", balance)

-----
```